



SETUP GUIDE

NVIDIA Triton Inference Server — Local Environment

Purpose: Set up NVIDIA Triton Inference Server locally using Docker, create a model repository, test REST and gRPC inference, and verify dynamic batching

SECTION 1: PREREQUISITES

Before starting, ensure you have:

- Python Docker installed and running (from Week 4 setup)
- At least 12GB free disk space (Triton image is ~10GB)
- Python 3.8+ with pip
- (Optional) NVIDIA GPU with drivers 525+ for GPU inference
- (Optional) nvidia-container-toolkit for GPU passthrough

If you do NOT have an NVIDIA GPU:

- Triton runs in CPU-only mode (slower but fully functional for learning)
- Skip GPU-specific steps marked with [GPU ONLY]

SECTION 2: PLATFORM-SPECIFIC SETUP

2A: WINDOWS (WSL2)

Step 1: Verify WSL2 and Docker

```
# Check WSL version
wsl --list --verbose
# Should show VERSION 2 for your distro
```

```
# Verify Docker
docker --version
docker run hello-world
```

Step 2: [GPU ONLY] Verify GPU in WSL2

```
# Check NVIDIA driver (Windows driver handles WSL2 GPU)
nvidia-smi
# Should show your GPU model and driver version 525+
```

```
# Verify Docker can see GPU
```

```
docker run --rm --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi
# Should show the same GPU info inside the container
```

Step 3: [GPU ONLY] Install nvidia-container-toolkit (if not already)

```
# Add NVIDIA container toolkit repository
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o
/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/$distribution/libnvidia-container.list | \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

# Install
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit

# Configure Docker runtime
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Step 4: Create workspace directory

```
mkdir -p ~/triton-lab
cd ~/triton-lab
```

2B: macOS

Note: macOS does not support NVIDIA GPUs in Docker. Triton will run in CPU-only mode. This is fine for learning configuration and API concepts.

Step 1: Verify Docker Desktop

```
docker --version
docker run hello-world
```

Step 2: Ensure Docker has sufficient resources

- Docker Desktop → Settings → Resources
- Set Memory: 8GB minimum (recommended 12GB)
- Set Disk: 20GB minimum

Step 3: Create workspace directory

```
mkdir -p ~/triton-lab
cd ~/triton-lab
```



2C: LINUX (Native)

Step 1: Verify Docker

```
docker --version
```

```
docker run hello-world
```

Step 2: [GPU ONLY] Verify NVIDIA driver and toolkit

```
# Check driver
```

```
nvidia-smi
```

```
# Should show driver 525+
```

```
# Install nvidia-container-toolkit if needed
```

```
sudo apt-get update
```

```
sudo apt-get install -y nvidia-container-toolkit
```

```
sudo nvidia-ctk runtime configure --runtime=docker
```

```
sudo systemctl restart docker
```

```
# Verify GPU in Docker
```

```
docker run --rm --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi
```

Step 3: Create workspace directory

```
mkdir -p ~/triton-lab
```

```
cd ~/triton-lab
```

SECTION 3: PULL TRITON DOCKER IMAGE

```
cd ~/triton-lab
```

```
# Pull the Triton Inference Server image
```

```
# Use a specific version for reproducibility (24.08 = August 2024 release)
```

```
docker pull nvcr.io/nvidia/tritonserver:24.08-py3
```

```
# Verify the image
```

```
docker images | grep tritonserver
```

```
# Should show ~10GB image
```

Expected output:

```
nvcr.io/nvidia/tritonserver 24.08-py3 abc123def456 X days ago 10.2GB
```

Note: The pull may take 10-30 minutes depending on your internet speed.

The image is large because it includes ALL backend frameworks.

SECTION 4: CREATE MODEL REPOSITORY

Step 1: Create directory structure

```
cd ~/triton-lab
# Create the model repository
mkdir -p model_repository/simple_onnx_model/1
```

Step 2: Download a sample ONNX model

```
# Download a small DenseNet model from ONNX Model Zoo
# (~30MB, image classification)
wget -O model_repository/simple_onnx_model/1/model.onnx \
  https://github.com/onnx/models/raw/main/validated/vision/classification/densenet-121/model/
densenet-12.onnx
```

```
# If wget not available, use curl:
# curl -L -o model_repository/simple_onnx_model/1/model.onnx \
# https://github.com/onnx/models/raw/main/validated/vision/classification/densenet-121/
model/densenet-12.onnx
```

Alternative (RECOMMENDED): Create a minimal custom ONNX model:

```
# Install onnx in Python (if not already from Week 6)
pip install onnx numpy
```

```
# Create a minimal model
python3 -c "
import onnx
from onnx import helper, TensorProto
```

```
# Simple identity model for testing
# NOTE: Use None for first dim (dynamic batch) — required when max_batch_size > 0
# NOTE: Set ir_version=8 — newer onnx libraries default to IR 13 which
# Triton 24.08's bundled ONNX Runtime does not support (max IR 10)
X = helper.make_tensor_value_info('input', TensorProto.FLOAT, [None, 3, 224, 224])
Y = helper.make_tensor_value_info('output', TensorProto.FLOAT, [None, 3, 224, 224])
node = helper.make_node('Identity', ['input'], ['output'])
graph = helper.make_graph([node], 'test_graph', [X], [Y])
model = helper.make_model(graph, opset_imports=[helper.make_opsetid("", 13)])
model.ir_version = 8
onnx.save(model, 'model_repository/simple_onnx_model/1/model.onnx')
```

```
print('Model created successfully')  
"
```

Step 3: Create model configuration

```
cat > model_repository/simple_onnx_model/config.pbtxt << 'EOF'  
name: "simple_onnx_model"  
platform: "onnxruntime_onnx"  
max_batch_size: 8  
input [  
  {  
    name: "input"  
    data_type: TYPE_FP32  
    dims: [3, 224, 224]  
  }  
]  
output [  
  {  
    name: "output"  
    data_type: TYPE_FP32  
    dims: [3, 224, 224]  
  }  
]  
dynamic_batching {  
  max_queue_delay_microseconds: 100000  
  preferred_batch_size: [2, 4, 8]  
}  
EOF
```

Step 4: Verify repository structure

```
tree model_repository/  
# Expected:  
# model_repository/
```

If `tree` is not installed: `ls -R model\_repository/`

SECTION 5: LAUNCH TRITON SERVER

5A: WITH GPU

```
docker run --rm --gpus all \  
-p 8000:8000 \  

```



```
-p 8001:8001 \  
-p 8002:8002 \  
-v $(pwd)/model_repository:/models \  
nvcr.io/nvidia/tritonserver:24.08-py3 \  
tritonserver --model-repository=/models
```

5B: WITHOUT GPU (CPU-only)

```
docker run --rm \  
-p 8000:8000 \  
-p 8001:8001 \  
-p 8002:8002 \  
-v $(pwd)/model_repository:/models \  
nvcr.io/nvidia/tritonserver:24.08-py3 \  
tritonserver --model-repository=/models
```

Expected Startup Output:

```
I0101 00:00:00.000000 1 server.cc:XXX]  
+-----+-----+-----+  
| Model           | Version | Status  |  
+-----+-----+-----+  
| simple_onnx_model | 1       | READY   |  
+-----+-----+-----+  
...  
I0101 00:00:00.000000 1 grpc_server.cc:XXX] Started GRPCInferenceService at 0.0.0.0:8001  
I0101 00:00:00.000000 1 http_server.cc:XXX] Started HTTPService at 0.0.0.0:8000  
I0101 00:00:00.000000 1 http_server.cc:XXX] Started Metrics Service at 0.0.0.0:8002
```

Key: Look for "READY" status for your model and all three services started.

SECTION 6: TEST INFERENCE ENDPOINTS

Open a NEW terminal (keep Triton running in the first terminal).

Step 1: Health Check

```
# Server liveness  
curl -s localhost:8000/v2/health/live  
# Expected: (empty 200 response)  
  
# Server readiness  
curl -s localhost:8000/v2/health/ready  
# Expected: (empty 200 response)
```

```
# Model readiness
curl -s localhost:8000/v2/models/simple_onnx_model/ready
# Expected: (empty 200 response)
```

Step 2: Model Metadata

```
curl -s localhost:8000/v2/models/simple_onnx_model | python3 -m json.tool
# Expected: JSON with model name, versions, platform, inputs, outputs
```

Step 3: REST Inference Request

```
# Install Python client for sending properly formatted requests
pip install tritonclient[all] numpy

# Send inference request
python3 -c "
import numpy as np
import tritonclient.http as httpclient

# Create client
client = httpclient.InferenceServerClient(url='localhost:8000')

# Check server is ready
print('Server ready:', client.is_server_ready())
print('Model ready:', client.is_model_ready('simple_onnx_model'))

# Create input data (random image tensor)
input_data = np.random.randn(1, 3, 224, 224).astype(np.float32)

# Create input/output objects
inputs = [httpclient.InferInput('input', input_data.shape, 'FP32')]
inputs[0].set_data_from_numpy(input_data)
outputs = [httpclient.InferRequestedOutput('output')]

# Send inference request
result = client.infer('simple_onnx_model', inputs, outputs=outputs)
output_data = result.as_numpy('output')
print('Output shape:', output_data.shape)
print('Inference successful!')
"
```

Step 4: gRPC Inference Request

```
python3 -c "
```

```
import numpy as np
import tritonclient.grpc as grpcclient

# Create gRPC client
client = grpcclient.InferenceServerClient(url='localhost:8001')

# Check server
print('Server ready:', client.is_server_ready())

# Create input
input_data = np.random.randn(1, 3, 224, 224).astype(np.float32)
inputs = [grpcclient.InferInput('input', input_data.shape, 'FP32')]
inputs[0].set_data_from_numpy(input_data)
outputs = [grpcclient.InferRequestedOutput('output')]

# Send inference
result = client.infer('simple_onnx_model', inputs, outputs=outputs)
output_data = result.as_numpy('output')
print('gRPC Output shape:', output_data.shape)
print('gRPC inference successful!')
"
```

Step 5: Check Metrics

```
# Get Prometheus metrics
curl -s localhost:8002/metrics | grep "nv_inference"
# Should show inference count, latency, queue time metrics
```

SECTION 7: VERIFY DYNAMIC BATCHING

```
python3 -c "
import numpy as np
import tritonclient.http as httpclient
from concurrent.futures import ThreadPoolExecutor
import time

client = httpclient.InferenceServerClient(url='localhost:8000')

def send_request(request_id):
    input_data = np.random.randn(1, 3, 224, 224).astype(np.float32)
    inputs = [httpclient.InferInput('input', input_data.shape, 'FP32')]
```



```
inputs[0].set_data_from_numpy(input_data)
outputs = [httpclient.InferRequestedOutput('output')]
start = time.time()
result = client.infer('simple_onnx_model', inputs, outputs=outputs)
elapsed = time.time() - start
return request_id, elapsed

# Send 16 requests concurrently to trigger batching
print('Sending 16 concurrent requests to trigger dynamic batching...')
with ThreadPoolExecutor(max_workers=16) as executor:
    futures = [executor.submit(send_request, i) for i in range(16)]
    results = [f.result() for f in futures]

for req_id, elapsed in sorted(results):
    print(f' Request {req_id:2d}: {elapsed*1000:.1f} ms')

print()
print('If dynamic batching is working, requests should complete in')
print('similar times (batched together) rather than sequentially.')
print()

# Check batch metrics
import urllib.request
metrics = urllib.request.urlopen('http://localhost:8002/metrics').read().decode()
for line in metrics.split('\n'):
    if 'batch_size' in line.lower() and 'simple_onnx' in line:
        print(line)
"
```

SECTION 8: ADD A SECOND MODEL (MULTI-MODEL SERVING)

Step 1: Create a second model (simple Python backend)

```
mkdir -p model_repository/echo_model/1

# Create a Python backend model that echoes input
cat > model_repository/echo_model/1/model.py << 'EOF'
import triton_python_backend_utils as pb_utils
import numpy as np

class TritonPythonModel:
    def initialize(self, args):
```

pass

```
def execute(self, requests):
```

```
    responses = []
```

```
    for request in requests:
```

```
        input_tensor = pb_utils.get_input_tensor_by_name(request, "INPUT")
```

```
        output_tensor = pb_utils.Tensor("OUTPUT", input_tensor.as_numpy())
```

```
        response = pb_utils.InferenceResponse(output_tensors=[output_tensor])
```

```
        responses.append(response)
```

```
    return responses
```

```
def finalize(self):
```

```
    pass
```

```
EOF
```

```
# Create config
```

```
cat > model_repository/echo_model/config.pbtxt << 'EOF'
```

```
name: "echo_model"
```

```
backend: "python"
```

```
max_batch_size: 4
```

```
input [
```

```
{
```

```
    name: "INPUT"
```

```
    data_type: TYPE_FP32
```

```
    dims: [10]
```

```
}
```

```
]
```

```
output [
```

```
{
```

```
    name: "OUTPUT"
```

```
    data_type: TYPE_FP32
```

```
    dims: [10]
```

```
}
```

```
]
```

```
instance_group [
```

```
{
```

```
    count: 1
```

```
    kind: KIND_CPU
```

```
}
```

```
]
```

```
EOF
```

Step 2: Restart Triton (it will detect both models)

```
# Stop the running container (Ctrl+C) and restart
# Or use Triton's model control API if enabled

# Restart with same command from Section 5
# Look for BOTH models showing "READY" in startup output:
# +-----+-----+-----+
# | Model           | Version | Status |
# +-----+-----+-----+
# | echo_model      | 1       | READY  |
# | simple_onnx_model | 1       | READY  |
# +-----+-----+-----+
```

Step 3: Test multi-model serving

```
python3 -c "
import numpy as np
import tritonclient.http as httpclient

client = httpclient.InferenceServerClient(url='localhost:8000')

# Test echo_model
input_data = np.array([[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]], dtype=np.float32)
inputs = [httpclient.InferInput('INPUT', input_data.shape, 'FP32')]
inputs[0].set_data_from_numpy(input_data)
outputs = [httpclient.InferRequestedOutput('OUTPUT')]

result = client.infer('echo_model', inputs, outputs=outputs)
output = result.as_numpy('OUTPUT')
print('Echo model input: ', input_data[0])
print('Echo model output:', output[0])
print('Multi-model serving verified!')
"
```

SECTION 9: MODEL VERSIONING

Step 1: Add a second version of the ONNX model

```
# Create version 2 directory
mkdir -p model_repository/simple_onnx_model/2

# Copy model (in production, this would be a retrained model)
cp model_repository/simple_onnx_model/1/model.onnx \
  model_repository/simple_onnx_model/2/model.onnx
```

Step 2: Configure version policy (optional)

Add to config.pbtxt to control which versions are served:

Serve only the latest version:

```
version_policy { latest { num_versions: 1 }}
```

Serve all versions:

```
version_policy { all {} }
```

Serve specific versions:

```
version_policy { specific { versions: [1, 2] }}
```

Step 3: Restart and verify both versions

After restart, check model versions

```
curl -s localhost:8000/v2/models/simple_onnx_model | python3 -m json.tool
```

Should show version "1" and "2" in the versions list

SECTION 10: TROUBLESHOOTING

ISSUE: Docker commands produce no output in WSL (silent failure)

CAUSE: WSL session was opened before Docker Desktop started, or Docker Desktop was restarted/updated after WSL was launched

FIX:

- Close ALL WSL terminal windows completely
- Ensure Docker Desktop is running (green "Engine running" at bottom)
- Open a fresh WSL terminal
- Verify: `docker --version` (should print version number)
- If still silent: Docker Desktop → Settings → Resources → WSL Integration

→ ensure your Ubuntu distro toggle is ON → Apply & restart

ISSUE: "WARNING: Detected GPU... not yet supported" + "ERROR: No supported GPU(s)"

CAUSE: Newer GPU (e.g., Blackwell/RTX PRO 2000) not recognized by Triton 24.08

FIX:

- This is a WARNING only — Triton still runs and serves models on the GPU.
- The model will load and inference works normally. Ignore this message.
- The repeating "Unable to get power limit" warnings are also harmless.

ISSUE: "Unsupported model IR version: 13, max supported IR version: 10"

CAUSE: Newer onnx Python library (1.17+) creates models with IR version 13, but Triton 24.08's bundled ONNX Runtime only supports up to IR 10

FIX:

- Set `ir_version=8` when creating the ONNX model:



- `model.ir_version = 8`

(Already done in the RECOMMENDED model creation step above)

ISSUE: "for the model to support batching the first dimension must be -1"

CAUSE: Model has fixed batch dimension [1,3,224,224] but config has `max_batch_size > 0`

FIX:

- Create model with dynamic first dimension: [None, 3, 224, 224]

(Already done in the RECOMMENDED model creation step above)

ISSUE: Docker cannot find GPU

Verify nvidia-container-toolkit is installed

`nvidia-ctk --version`

Check Docker runtime configuration

`cat /etc/docker/daemon.json`

Should contain "nvidia" runtime

Restart Docker

`sudo systemctl restart docker`

ISSUE: Port already in use

Find what's using the port

`sudo lsof -i :8000`

Or

`sudo netstat -tlnp | grep 8000`

Kill the process or remap ports:

`docker run ... -p 9000:8000 -p 9001:8001 -p 9002:8002 ...`

Then access via localhost:9000

ISSUE: Model fails to load (shape mismatch)

Check Triton logs for error details

Common: config dims don't match actual model input/output shapes

Verify model shape with Python:

```
python3 -c "
```

```
import onnx
```

```
model = onnx.load('model_repository/simple_onnx_model/1/model.onnx')
```

```
for inp in model.graph.input:
```

```
    print(f'Input: {inp.name}, Shape: {[d.dim_value for d in inp.type.tensor_type.shape.dim]}')
```

```
for out in model.graph.output:
```

```
    print(f'Output: {out.name}, Shape: {[d.dim_value for d in out.type.tensor_type.shape.dim]}')
```

```
"
```

Update config.pbtxt dims to match these shapes

ISSUE: Out of Memory (GPU)

Check GPU memory usage

nvidia-smi

Options:

1. Use smaller model

2. Reduce max_batch_size in config.pbtxt

3. Use CPU-only mode (remove --gpus all)

4. Enable model loading on CPU with GPU compute:

Add to config: instance_group [{ kind: KIND_GPU, count: 1 }]

ISSUE: Triton image pull fails or is very slow

Use a mirror or specific tag

docker pull nvcr.io/nvidia/tritonserver:24.08-py3

If NGC login required:

docker login nvcr.io

Username: \$oauthtoken

Password: <your NGC API key from <https://ngc.nvidia.com>>

ISSUE: tritonclient installation fails

Install with specific extras

pip install tritonclient[http] # HTTP only

pip install tritonclient[grpc] # gRPC only

pip install tritonclient[all] # Both

If grpc fails on ARM/macOS:

pip install tritonclient[http] # Use HTTP client only

SECTION 11: VERIFICATION CHECKLIST

- Docker running and can pull images
- Triton image pulled (~10GB)
- Model repository created with correct structure
- ONNX model file present in version directory
- config.pbtxt created with correct input/output dims
- Triton server starts without errors
- Model shows "READY" in startup log
- Health endpoint returns 200 (/v2/health/ready)



- REST inference returns valid output
- gRPC inference returns valid output
- Metrics endpoint accessible (/metrics)
- Dynamic batching verified (concurrent requests batch together)
- [Optional] Second model (echo_model) added and serving
- [Optional] Model versioning configured and working
- [GPU ONLY] GPU visible in Docker (nvidia-smi works inside container)

SECTION 12: QUICK REFERENCE — DAILY COMMANDS

```
# Navigate to workspace  
cd ~/triton-lab
```

```
# Start Triton (GPU)  
docker run --rm --gpus all -p 8000:8000 -p 8001:8001 -p 8002:8002 \  
-v $(pwd)/model_repository:/models \  
nvcr.io/nvidia/tritonserver:24.08-py3 tritonserver --model-repository=/models
```

```
# Start Triton (CPU-only)  
docker run --rm -p 8000:8000 -p 8001:8001 -p 8002:8002 \  
-v $(pwd)/model_repository:/models \  
nvcr.io/nvidia/tritonserver:24.08-py3 tritonserver --model-repository=/models
```

```
# Health check  
curl -s localhost:8000/v2/health/ready && echo "READY" || echo "NOT READY"
```

```
# List models  
curl -s localhost:8000/v2/models | python3 -m json.tool
```

```
# Model info  
curl -s localhost:8000/v2/models/simple_onnx_model | python3 -m json.tool
```

```
# View metrics  
curl -s localhost:8002/metrics | grep nv_inference
```

```
# Stop Triton  
docker ps          # Find container ID  
docker stop <container_id>  # Or just Ctrl+C in the terminal
```

```
# Clean up
```



docker system prune

Remove unused images/containers

SECTION 13: NEXT STEPS (OPTIONAL EXPLORATION)

Once the basic setup works, try these:

1. Export your own PyTorch model to ONNX and serve it:

```
import torch
model = YourModel()
dummy_input = torch.randn(1, ...)
torch.onnx.export(model, dummy_input, "model.onnx")
```

2. Use Perf Analyzer for benchmarking:

```
docker run --rm --net=host \
  nvcr.io/nvidia/tritonserver:24.08-py3-sdk \
  perf_analyzer -m simple_onnx_model -u localhost:8000 \
  --concurrency-range 1:16:2
```

3. Set up model ensemble (chain two models):

- Create preprocessor model + main model
- Define ensemble config linking them

4. Enable explicit model control:

```
tritonserver --model-repository=/models --model-control-mode=explicit
# Then load/unload models via Management API:
# POST localhost:8000/v2/repository/models/model_name/load
# POST localhost:8000/v2/repository/models/model_name/unload
```